IMPLEMENTATION PLAN

EasyShop

Client App, 5-Day Plan

A B C

TEAM	3 engineers, one GitHub repo. **A**, **B**, **C** are placeholders. Replace them with real names before sending.
DURATION	5 days. This is shorter than the Admin week because `core` already exists and the Admin app proved the patterns.
STARTS	Right after the Admin app is finished and merged into `main`. Nothing from the Admin week is carried into this plan, so day 1 is a full Client day for everyone.
TASK IDS	Every task starts with `CL-` (for example `CL-A3`), so they do not clash with the Admin task IDs in `status.md`.

> Time estimates assume about **6 focused hours per day**. Most days are between 5.5 and 6.5 hours, so there is some room for reviews. If a task runs over by more than half a day, tell the team the same day.

The team at a glance

Auth, Session, Profile, README

Person A | about 28h

Client auth (register, login, OTP, verified), session, route guard, logout. Profile (view and edit). README.

Client theme, Catalog, Favorites, Orders screens

Person B | about 28.5h

Client theme. Home, categories, products, product details. Favorites. Orders list and details screens.

Shell, Cart, Checkout, Orders data, Release

Person C | about 30h

Client shell (entry point, router, injection). Cart. Checkout. Orders domain and data. Payment success. Client APK.

1 Starting point

This plan assumes the Admin app is finished. Before day 1 it should meet its own definition of done: all features merged into `develop`, tests written, the admin APK built, the Admin README written, and `develop` merged into `main`. If any Admin item is still open when the Client week starts, finish it first and do not mix it into this plan.

Ready to reuse from `core` (do not rebuild):

- Network and errors: `DioConsumer`, `AuthInterceptor`, `guardedCall` / `guardedUnitCall`, `ResponseReader`, `multipartFileOrNull`, `json_parsers`, `SecureStorageHelper`, `NetworkInfo`
- UI: theme, `AppButton`, `AppTextField`, `OtpInput`, `AppLoading` / `AppEmpty` / `AppError`, `confirm_delete_dialog`, `AppNetworkImage`, `AppCard`, `AppAvatar`, `AppIconBadge`, `AppStatusLabel`
- Helpers: `PriceFormatter`, `AppDateFormatter`, `PhoneUtils`, `Validators`, `ImagePickerService`
- Maps: `LocationPickerScreen`, `LocationViewerScreen`, `OsmMap`, `LocationService`, `AddressResolver`
- Tests: `FakeApiConsumer` in `test/helpers`

Not ready yet for the Client app:

- `main_client.dart` does not call `initCore` yet.
- **No Client response shapes are documented.** Day 1 Postman fixes this.
- The Client theme (the Client Figma uses a different orange, about `#FF6900`) does not exist yet.

No new packages are planned for the Client app.

2 What we are building (Client app)

From the brief and the Client Figma:

- **Auth:** register (one screen), OTP (6 digits, resend), "Verified!" screen, login, session restore, logout
- **Home:** greeting, search, category chips, products grid, "Out of Stock" badge
- **Product details:** quantity selector, Add to Cart
- **Favorites:** list, empty state, add and remove
- **Cart:** items, quantity change, remove, total
- **Checkout:** delivery address on a map (latitude and longitude sent), payment method (cash or card), place order
- **Payment successful** screen

- **Orders:** list and details with status
- **Profile:** view and edit (with location on a map), logout

Flow required by the brief: **Home, Categories / Products, Product Details, Add to Cart, Checkout, Select Address on Map, Place Order, Order Details.**

3 Who owns what

Why this split: A already built the Admin auth, so the Client auth is the same pattern. B owns the catalog and the Client look, because B built the shared UI. C owns the buying flow (cart, checkout, orders), which is one chain where each step feeds the next, and which uses the map picker C built.

4 Day-by-day overview

Day	A	B	C
1	Postman: client auth + profile. Client auth domain + data	Client theme. Postman: catalog + favorites. Categories + products domain + data (start)	Postman: cart, orders, payment. Client shell. Cart domain + data (start, **publish the contract at the end of the day**)
2	Login screen. OTP + Verified screen (**real token by the end of the day**)	Catalog data (finish). Home screen	Cart data (finish). Cart screen
3	Register screen. Session restore, guard, logout	Product details. Favorites (start)	Cart screen (finish). Orders domain + data (**publish early**). Checkout (start)
4	Profile domain + data and view. Auth tests. Edit profile (start)	Favorites (finish). Orders list + details	Checkout (finish). Payment success screen. Tests (start)
5	Edit profile (finish). Profile tests. README (both apps)	Tests. Responsive check and Figma polish	Tests. Client APK. Final merge to `main`

5 Task tables per person

How to read the columns

- **Est.** is the estimated focused time.
- **Blocked by** means the task cannot be finished (or sometimes started) until that task or decision is done.
- **Unblocks** lists who is waiting on it.
- While a blocker is unfinished, build against a **fake repository** (a temporary, uncommitted preview, as in the Admin week) and swap in the real one later.

A Auth, Session, Profile, README

about 28h of work

Branches: `feature/client-auth`, `feature/client-session`, `feature/client-profile`

ID	Task	Day	Est.	Blocked by	Unblocks
CL-A1	Postman: client register, OTP verify and resend, login, logout, profile, profile update. Document in `docs/client-api-shapes.md`: token field, error format, **does register send an OTP, does login send an OTP, what does verify return**, and whether `PUT` with form-data needs `POST` + `_method=PUT`	1	**2h**	API key and base URL (local only)	CL-A2, CL-A4, CL-A5, CL-A7
CL-A2	Client auth domain + data: params, use cases (register, login, verify, resend, logout), remote data source (register uses `FormData`), repository with `guardedCall`, `registerClientAuth(sl)`. Start with the domain layer while waiting for the shell	1	**3.5h**	CL-A1, CL-C2 (shell and injection, same day)	CL-A3, CL-A4, CL-A5, CL-A6
CL-A3	Login screen + `LoginCubit` (validation, no double submit, callbacks for navigation)	2	**2.5h**	CL-A2 (or a fake), CL-B1 (theme; use the Admin theme until it merges)	CL-A4
CL-A4	OTP screen + `OtpCubit` (6 digits, resend with "OTP resent successfully!", save token) and the "Verified!" screen	2	**3.5h**	CL-A1 (OTP behavior), CL-A2, CL-A3	**A real token for B and C by the end of day 2**, CL-A6
CL-A5	Register screen + `RegisterCubit`: one screen, optional address, optional location through `LocationPickerScreen`, phone with `PhoneUtils`, validation	3	**3h**	CL-A2	
CL-A6	Session: `SessionCubit`, restore on app start, route guard in `client_router.dart`, logout	3	**2.5h**	CL-C2 (guard skeleton), CL-A4	All protected screens, CL-A7
CL-A7	Profile domain + data and the profile screen (Personal Information, Delivery Address, My Orders, Favorites, Edit Profile, Logout)	4	**3h**	CL-A1, CL-A6	CL-A8, CL-C6 (saved address), CL-B4 (greeting name)
CL-A8	Edit profile: form, `POST` + `_method=PUT` with `FormData`, location update on a map	4-5	**3h**	CL-A7	
CL-A9	Tests: auth use cases, repository, login / OTP / register Cubits (day 4, 1.5h). Profile use cases, models, Cubits (day 5, 1.5h)	4-5	**3h**	CL-A2-A7	
CL-A10	README for both apps: idea, how to run both flavors, architecture, main packages, **honest list of incomplete features**	5	**2h**	CL-C9 (run commands), final package list	

B Client theme, Catalog, Favorites, Orders screens

about 28.5h of work

Branches: `feature/client-theme`, `feature/client-catalog`, `feature/client-favorites`, `feature/client-orders-screens`

ID	Task	Day	Est.	Blocked by	Unblocks
CL-B1	Client theme: Client colors (about `#FF6900`) in `core/theme`, Client section in `docs/design-tokens.md`	1	**2h**	Exact color values from the Client Figma export	CL-A3, CL-C2, all Client UI
CL-B2	Postman: client categories, products, product details, favorites (GET, store, destroy with a body). Document stock, image, seller name and favorite shape in `docs/client-api-shapes.md`	1	**1h**	API key and base URL	CL-B3, CL-B6
CL-B3	Domain + data for `features/client/categories` and `features/client/products`: own entities, use cases (`GetCategories`, `GetProducts`, `GetProductDetails`), tolerant models, remote data sources, repositories, `registerClientCatalog(sl)`	1-2	**4h**	CL-B2, CL-C2 (injection, same day)	CL-B4, CL-B5
CL-B4	Home screen: greeting (takes `userName`, null until CL-A7, like `adminName` in Admin), search, category chips, products grid, "Out of Stock" badge, loading / empty / error states, pull to refresh. Search and category filter are **client-side** (no API support)	2	**5h**	CL-B3, CL-B1, CL-C2	CL-B5, CL-B6
CL-B5	Product details: image, price, seller, description, stock, quantity selector (not above stock), Add to Cart with "Added to Cart!" feedback. Add to Cart is disabled when out of stock	3	**3.5h**	CL-B3, CL-C3 (cart contract, published at the end of day 1)	CL-B6
CL-B6	Favorites: domain + data and screen (empty state, list, error with retry), plus a heart on product details to add and remove (see decision 6)	3-4	**4h**	CL-B2, CL-B5	
CL-B7	Orders list and details screens: list with status, items, payment, total. Details: items, delivery with "View on Map" through `LocationViewerScreen`, payment, summary. Read-only	4	**4h**	CL-C5 (orders data, published on day 3), CL-B1	
CL-B8	Tests: catalog use cases, models, repositories, Cubits (Home, details), favorites Cubit, orders Cubits	5	**3h**	CL-B3-B7	
CL-B9	Responsive check and Figma polish on all Client screens	5	**2h**	All screens exist	

C Shell, Cart, Checkout, Orders data, Release

about 30h of work

Branches: `feature/client-shell`, `feature/client-cart`, `feature/client-orders-data`, `feature/client-checkout`

ID	Task	Day	Est.	Blocked by	Unblocks
CL-C1	Postman: client cart (GET, store, destroy with a body), orders (list, details, store), `orders/pay`, `payments/process`. **Create** `docs/client-api-shapes.md` and answer the open cart and order questions in section 7	1	**2h**	API key and base URL	CL-C3, CL-C4, CL-C5, CL-B5
CL-C2	Client shell: `main_client.dart` calls `AppConfig.load` and `initCore`, `client_app.dart`, `client_router.dart` (route names in one file, 5-tab bottom navigation with placeholder screens, guard skeleton), `client_injection.dart` (`initClient`). **Check where** `isJwtExpired` **lives; if it is inside** `features/admin`, **move it to** `core` **first** (small PR), so the Client can reuse it	1	**3h**	CL-B1 (use the Admin theme until it merges)	CL-A2, CL-A6, CL-B3, everyone (to run the app)
CL-C3	Cart domain + data: entities, contract, use cases (`GetCart`, `AddToCart`, `RemoveFromCart`), models, remote data source (`DELETE` with a body), repository, `registerClientCart(sl)`. **Publish the contract at the end of day 1**	1-2	**3.5h**	CL-C1, CL-C2	CL-B5, CL-C4, CL-C6
CL-C4	Cart screen + `CartCubit`: items, quantity change, remove with the shared confirm dialog, total, empty state, "Proceed to Checkout", cart count on the tab	2-3	**4h**	CL-C3, CL-C1 (how quantity changes)	CL-C6
CL-C5	Orders domain + data: entities, contract, use cases (`PlaceOrder`, `GetOrders`, `GetOrderDetails`), request with parallel lists (`product_id`, `quantity`, `price`), models, remote data source, repository, `registerClientOrders(sl)`. Item prices come from the cart, never from a typed value. **Publish early**	3	**4h**	CL-C1, CL-C2	CL-C6, CL-B7
CL-C6	Checkout screen + `CheckoutCubit`: delivery address (saved address first, "Change Location" opens the map picker, latitude and longitude sent), payment method (cash or card), order summary, Place Order with no double submit	3-4	**5h**	CL-C3, CL-C4, CL-C5, CL-A7 (soft: saved address; the map picker works without it)	CL-C7
CL-C7	"Payment Successful!" screen: order code, View Orders, Continue Shopping	4	**2h**	CL-C6	
CL-C8	Tests: cart and orders use cases, models, repositories, Cubits, `CheckoutCubit` (including no double submit)	4-5	**4.5h**	CL-C3-C6	
CL-C9	Build the client APK (`--flavor client`), final `flutter analyze` and `dart format`, give run commands to A for the README, merge `develop` into `main`, walk through every Client screen against the Figma with the team	5	**2h**	Everything else	Delivery

ID	Task	Day	Est.	Blocked by	Unblocks
CL-C10	**Stretch, not counted:** card payment through `orders/pay/{code}` and `payments/process` (Paymob). Day 5 is lightest for A and B, so one of them can take it if C is busy	5	**3h**	CL-C6, CL-C1 (only if Postman shows it works), decision 5	

6 Critical path and what to do if something slips

The shortest chain that everything depends on:

Day 1 Postman (CL-A1, CL-B2, CL-C1) then the Client shell (CL-C2) then the data layers (CL-A2, CL-B3, CL-C3, CL-C5) then a real token (CL-A4, end of day 2).

- Until a real token exists, test screens with fake repositories. Do not hardcode a token anywhere.
- If Postman results are late, build against fakes. Make the swap to the real repository the first task of the day it lands.
- If A is behind on day 2, move Register (CL-A5) to B on day 3 and push favorites (CL-B6) one day later. B has spare time on day 5.
- If B is behind on day 4, A takes the orders list screen on day 5 (A has the lightest day 5).
- Anyone blocked for more than 2 hours says so in the team chat right away.

Cut list, in this order, if the week gets tight:

1. Card payment (CL-C10, already a stretch)
2. Heart on the product details (keep the Favorites screen)
3. Search on Home (keep the category chips)
4. Profile picture upload
5. Tests beyond Cubits and use cases

> **Never cut:** Login, OTP, Home, Details, Cart, Checkout with map, Place Order, Orders list and details.

7 Decisions and checks for day 1

#	Item	Who	Why it matters
1	**Client response shapes** are not documented. Each person adds their own section to `docs/client-api-shapes.md` (to avoid conflicts, one section per person in their own PR).	A, B, C	All data layers
2	**Client OTP behavior.** The Admin register does not send an OTP, only login does. The Client Figma goes Register, then OTP, then "Verified!". Test: does Client register send an OTP, does login, and what does verify return?	A	CL-A2, CL-A4, CL-A5
3	**Cart rules.** How does the quantity change (does `POST` to `carts/store` replace or add)? What does `GET /client/carts` return (full product objects or ids)? Does placing an order empty the cart? Does `DELETE` with a body work from Dio?	C	CL-C3, CL-C4, CL-C6, CL-B5
4	**Order request.** `product_id`, `quantity` and `price` are parallel lists, and the app sends the price. In the Admin tests the server stored the price the app sent. Always use the cart's price. Also record which order statuses and which `payment` data appear.	C	CL-C5
5	**Card payment.** The Figma has "Credit / Debit Card", but the shape of `orders/pay` and `payments/process` is unknown. Try it in Postman on day 1. If it does not work, keep it out and say so in the README (the brief asks for honesty about broken endpoints).	C, team	CL-C6, CL-C10
6	**Favorites need an entry point.** The Figma has a Favorites tab but no heart icon. Proposal: a heart in the product details top bar, like the trash icon we added in the Admin app.	B, team	CL-B6
7	**Order details screen is not in the Client Figma** (only the list). Proposal: build it from the Admin order details pattern.	B, team	CL-B7
8	**Search and category filter.** The API has no search or filter parameters, so both are client-side. Also check whether `GET /client/products` is paginated.	B	CL-B4
9	**Client theme colors.** We need exact values from the Figma export (the Admin one is `#F5821F`, the Client one is about `#FF6900`).	B	CL-B1
10	**Deadline** is still not set.	Team	All

8 Working rules for the week

- **Branches:** one feature, one branch, one small PR into `develop`. Unfinished tasks are never merged half-done. Merge PRs with "Create a merge commit".
- **Client and Admin never import each other.** If the Client needs something that lives in an Admin feature, move it to `core` first in a small PR.
- **New repositories** use `guardedCall` or `guardedUnitCall`. **New data sources** use `ResponseReader` and `multipartFileOrNull`. Do not write private copies.
- **Screens** receive navigation as callbacks (like `onOpenDetails` in Admin). Texts go in a `<Feature>Strings` class, paths in `end_points.dart`, no magic numbers.
- **Every API screen** shows loading, empty, and error with retry. Submit buttons are disabled while a request runs.
- `core` **files:** tell the team before editing a shared file and keep the PR small.
- **Tests:** hand-written fakes, no `bloc_test` or `mocktail`. Check `states` with `expectLater(cubit.stream, emitsInOrder([...]))`.
- **Generated plugin registrants** are committed together with the `pubspec.yaml` change that causes them.
- **Secrets:** base URL, `x-api-key` and tokens never go in git or in a chat.
- **Before every PR:** `dart format .`, `flutter analyze`, `flutter test`, and check that the Client flavor builds (`flutter build apk --flavor client -t lib/main_client.dart`) and that the Admin flavor still builds.
- **Daily:** a 10-15 minute check-in. Each person says what is done, what is next and what is blocked.

9 Definition of done (Client app)

- [ ] Register, OTP, Login, Home works end to end, with a saved session and logout
- [ ] Categories, products, details, search and category filter
- [ ] Favorites: add, remove, list
- [ ] Cart with quantities, checkout with the address picked on a map, place order, success screen
- [ ] Orders list and details, with the delivery location shown on a map
- [ ] Profile view and edit, with location
- [ ] Loading, empty and error-with-retry states on every screen
- [ ] Tests for use cases, repositories and main Cubit states
- [ ] `dart format` clean, `flutter analyze` with no important warnings
- [ ] README and a Client APK. Any incomplete feature or broken endpoint is listed honestly.
- [ ] No secrets in the repo

10 Update for status.md

```
2026-10-31: Schedule: Client app plan is 5 days (Admin was 7) and starts right after the Admin app is finished, with no Admin carry-over.
Client plan: A = client auth + session + profile + README; B = client theme + catalog + favorites + orders screens; C = client shell + cart +
checkout + orders data + client APK. Details in easyshop-client-app-plan.md.
```